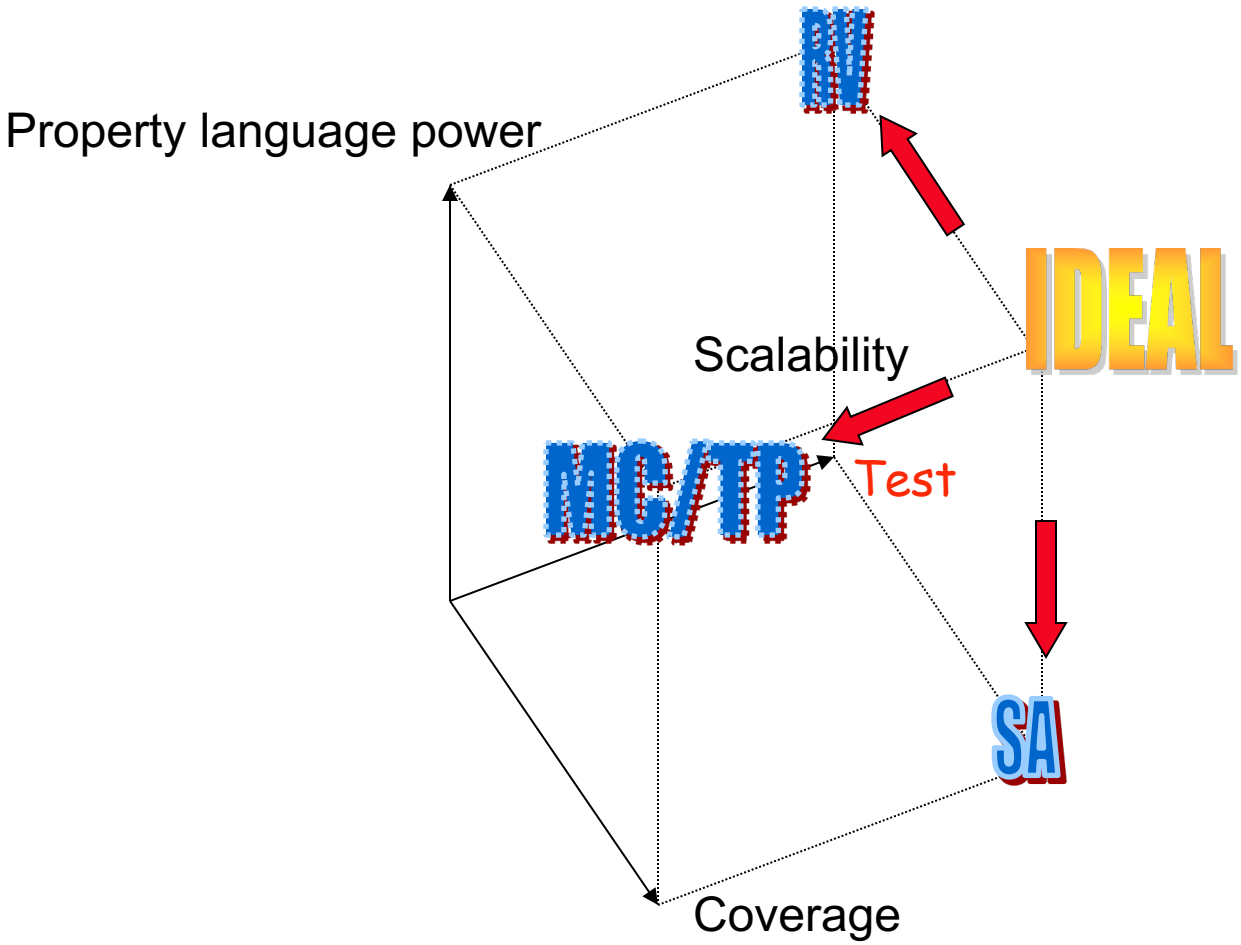
KESTREL TECHNOLOGY

It is not Ideal



KESTREL TECHNOLOGY

So Many Logics

- Assertions
- Pre-post conditions, invariants
- Temporal logic
- Time lines
- Automata
- Regular expressions
- Process algebra
- Mathematical languages
- Programming languages

KESTREL TECHNOLOGY

The Basic Problem

Given a finite trace/sequence of events:

e1 e1 e3 ….. eN

(the output of the instrumented program) …

specify as succinctly as possible required properties, and

check conformance between trace and specification as efficiently as possible.

Preferably on the fly

KESTREL TECHNOLOGY

Why can the specification not just be a program?

- It's worth a try.
- Functional programming seems to be suited due to its list processing orientation (trace = list).
- We will try with Specware, a specification/programming environment developed at Kestrel Institute.

KESTREL TECHNOLOGY

Specware

- A specification language
  - Allows a description to be abstract
  - Supports under specification, logic, set theory, abstract data types.
  - Supports program refinement, into:
- A functional programming language in the ML style
- We will use the functional programming style in this exercise

KESTREL TECHNOLOGY

Specware: Events and Traces

```
EVENT =
  spec
    type File = Nat
    type Data = Nat

  type Event =
          | open File
          | close File
          | read File * Data
          | write File *  Data
          | …

    type Trace = List Event
  endspec
```

Specware: Linear Temporal Logic (LTL)

```
LTL = spec
    import EVENT
☒ type Property = Trace -> Boolean

   op  always : Property -> Property
   def always P trace =
         case trace of
           Nil -> true
          |event :: trace1 ->  P trace && always P trace1

    op  until : Property * Property -> Property
    def until(P1,P2) trace =
         case trace of
           Nil -> false
          |event :: trace1 -> P2 trace || (P1 trace && until(P1,P2) trace1)

    op  eventually : Property -> Property
    def eventually P =   until(TRUE,P)
```

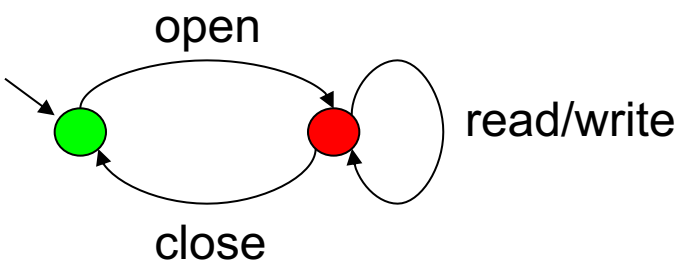
Specware: Properties 1+2
An open is followed by a close.

```
PROPERTIES_1_2 =
  spec
    import LTL

    def property1 : Property =
          always(isOpen IMPLIES (eventually isClose))
⊠⊠
    def property2 : Property =
          always(
            fn trace ->
              case trace of
                 (open f) :: _ -> eventually(first-event-is (close f)) trace
                |_ -> true
          )

endspec
```

Specware: Property 3 (open;(read|write)*;close)*




```
PROPERTY_3 =
  spec
    import LTL

    def property3 : Property = state1

    def state1 : Property =
          fn trace ->
            case trace of
              Nil -> true
             |(open _) :: trace1 -> state2(trace1)
             |_ -> false

  endspec
```

```
def  state2 : Property =
        fn trace ->
          case trace of
            Nil -> false
           |event :: trace1 ->
            (case event of
               read(_,_) -> state2 trace1
              |write(_,_) -> state2 trace1
              |close _ -> state1 trace1
              |_ -> false)
```

Analyzing a Trace

```
EXPERIMENT  =☒☒
spec
  import PROPERTIES_1_2
  import PROPERTY_3

  def file1 : File = 1
  def file2 : File = 2

  def trace : Trace = [open file1, write(file1,42), open file2, close file1]

  def result1 : Boolean = property1 trace -- succeeds
  def result2 : Boolean = property2 trace -- fails
  def result3 : Boolean = property3 trace -- fails

endspec
```

KESTREL TECHNOLOGY

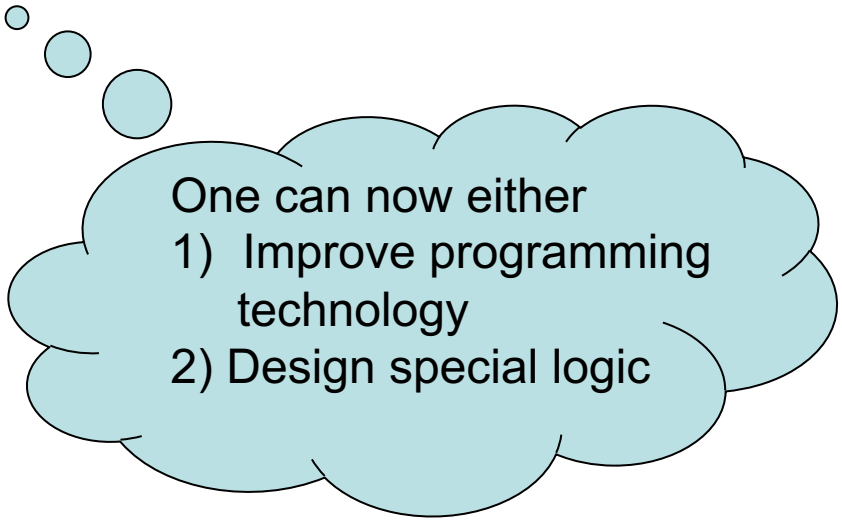
What was Useful?

- Ability to specify event data types
- Recursive definition of new combinators such as "always" and "eventually".
- Definition of state machines: a mixed notation where several "logics" can co-exist.
- Distinction between accept and reject states (what happens at the end of the trace)
- The capturing of data values which are then passed as arguments to functions. Pattern matching over occurring events seemed useful

KESTREL TECHNOLOGY

Problems

- The trace has to be available in its entirety
- The trace is traversed repeatedly



KESTREL TECHNOLOGY

Eagle

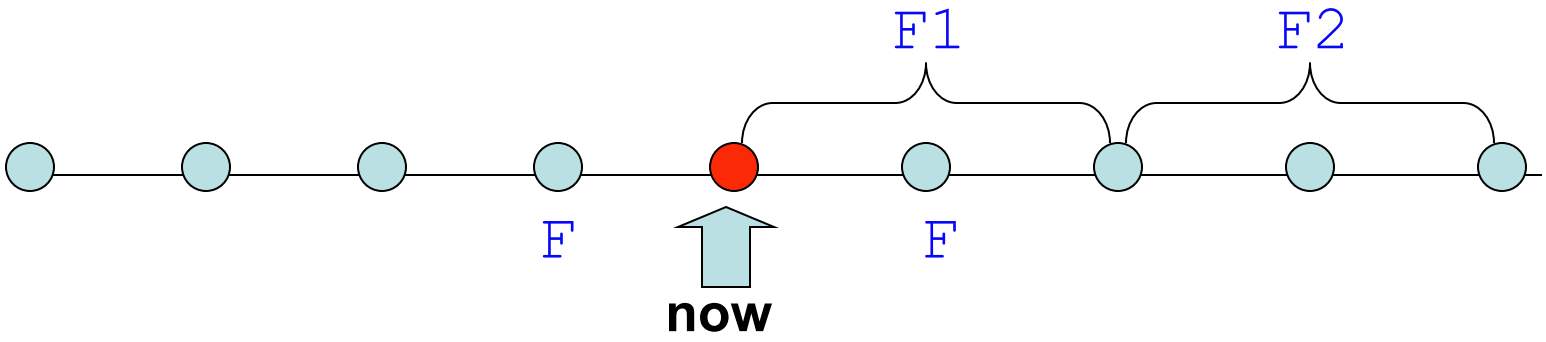


- Based on recursive parameterized rules over three “temporal connectives”: **next**, **previous** and **concatenation**.
- Has the flavor of functional trace programming.
- Can encode future time temporal logic, past-time logic, ERE, μ-calculus, real-time, data-binding, statistics….

Eagle's Core Concepts

- Prop. logic + Three temporal connectives:
  - **Next:** `@F`
  - **Previous:** `#F`
  - **Concatenation:** `F1;F2`
- Recursion:

```
always(Term t) = t /\ @always(t) .
```



KESTREL TECHNOLOGY

Syntax

$$
\begin{aligned}
S &::= \textbf{dec } D \textbf{ obs } O \\
D &::= R^* \\
O &::= M^* \\
R &::= \{\textbf{max} \mid \textbf{min}\}\ N(T_1\ x_1, \ldots, T_n\ x_n) = F \\
M &::= N = F \\
T &::= \textbf{Form} \mid \textit{java primitive type} \\
F &::= \textit{java expression} \mid \textbf{True} \mid \textbf{False} \mid \neg F \mid F_1 \wedge F_2 \mid F_1 \vee F_2 \mid F_1 \rightarrow F_2 \mid \\
&\quad \bigcirc F \mid \odot F \mid F_1 \cdot F_2 \mid N(F_1, \ldots, F_n)
\end{aligned}
$$

KESTREL TECHNOLOGY

Semantics

$\sigma, i \models_D exp$ iff $1 \leq i \leq |\sigma|$ and $evaluate(exp)(\sigma(i)) == true$

$\sigma, i \models_D \underline{\text{true}}$

$\sigma, i \not\models_D \underline{\text{false}}$

$\sigma, i \models_D \neg F$ iff $\sigma, i \not\models_D F$

$\sigma, i \models_D F_1 \wedge F_2$ iff $\sigma, i \models_D F_1$ and $\sigma, i \models_D F_2$

$\sigma, i \models_D F_1 \vee F_2$ iff $\sigma, i \models_D F_1$ or $\sigma, i \models_D F_2$

$\sigma, i \models_D F_1 \rightarrow F_2$ iff $\sigma, i \models_D F_1$ implies $\sigma, i \models_D F_2$

$\sigma, i \models_D \bigcirc F$ iff $i \leq |\sigma|$ and $\sigma, i+1 \models_D F$

$\sigma, i \models_D \odot F$ iff $1 \leq i$ and $\sigma, i-1 \models_D F$

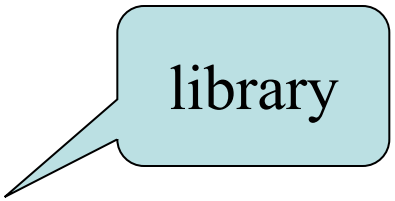
$\sigma, i \models_D F_1 \cdot F_2$ iff $\exists j$ s.t. $i \leq j \leq |\sigma|+1$ and $\sigma^{[1,j-1]}, i \models_D F_1$ and $\sigma^{[j,|\sigma|]}, 1 \models_D F_2$

$\sigma, i \models_D N(F_1, \ldots, F_m)$ iff

- if $1 \leq i \leq |\sigma|$ then:
  - $\sigma, i \models_D F[x_1 \mapsto F_1, \ldots, x_m \mapsto F_m]$
  - where $(N(T_1\ x_1, \ldots, T_m\ x_m) = F) \in D$
- otherwise, if $i = 0$ or $i = |\sigma|+1$ then:
  - rule N is defined as $\underline{\text{max}}$ in D

KESTREL TECHNOLOGY

Example




```
// LTL rules:

max always(Term t) = t /\ @ always(t) .

min eventually(Term t) = t \/ @ eventually(t) .

min previously(Term t) = t \/ # previously(t) .


// Monitors:

mon M1 = always(open() -> eventually(close())) .

mon M2 = always(close() -> previously(open())) .


mon M3 = always(open(file?) => eventually(close(file))) .

mon M4 = always(close(file?) => previously(open(file))) .
```

KESTREL TECHNOLOGY

Data Bindings

```
mon M3 = always(open(file?) => eventually(close(file))) .


// With an extra rule:

mon M3 = always(open() -> eventuallyClose(getFile())) .

min eventuallyClose(File file) = eventually(close(file)) .
```

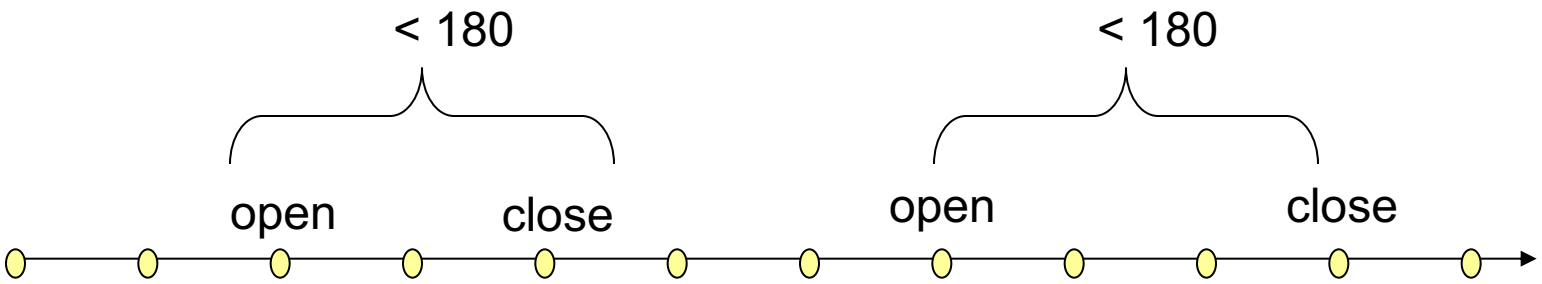
Time is Just Data

```
min Before(long t, Term F) =
      clock <= t /\
      (F \/ @ Before(t,F)) .

min Within(long t, Term F)  =
      Before(t+clock, F) .
```

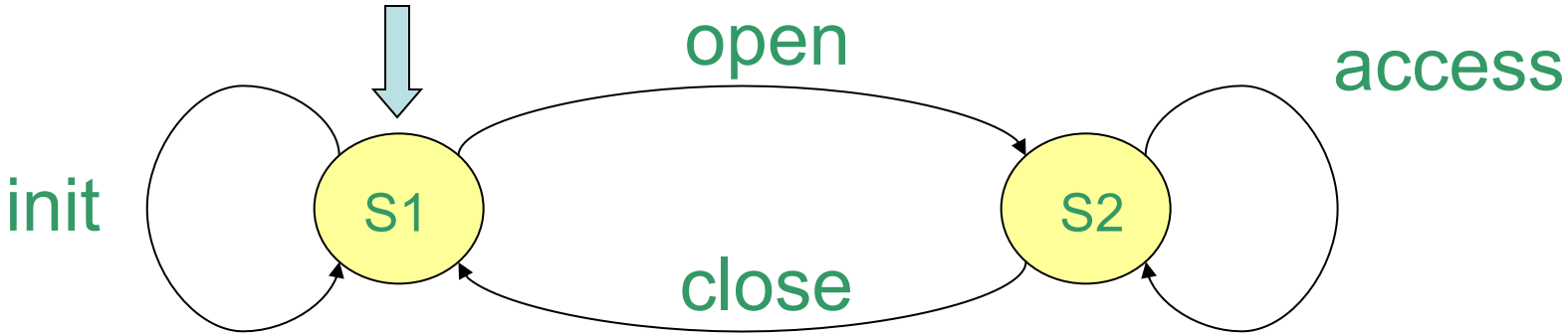
library

```
mon M = always(open() -> Within(180,close()))
```



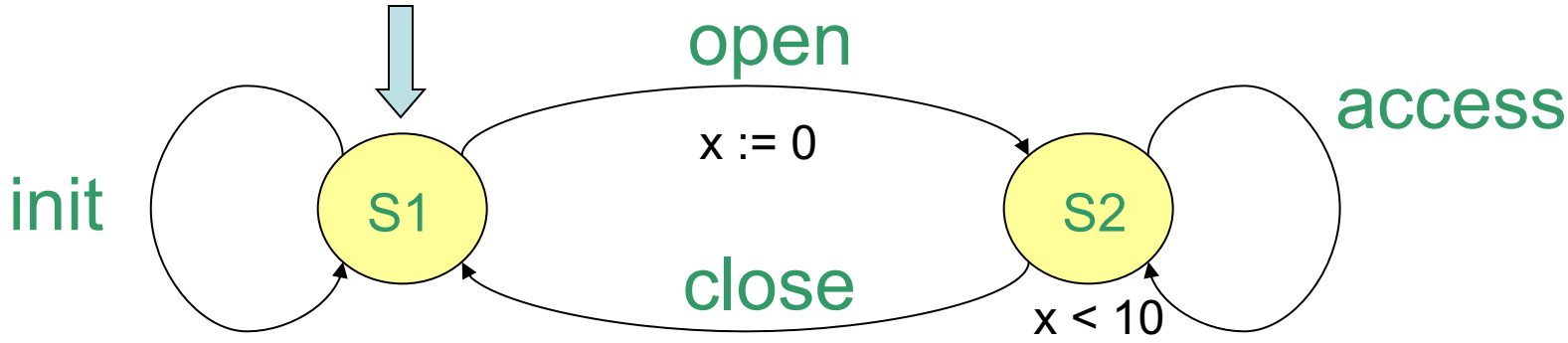
KESTREL TECHNOLOGY

Automata



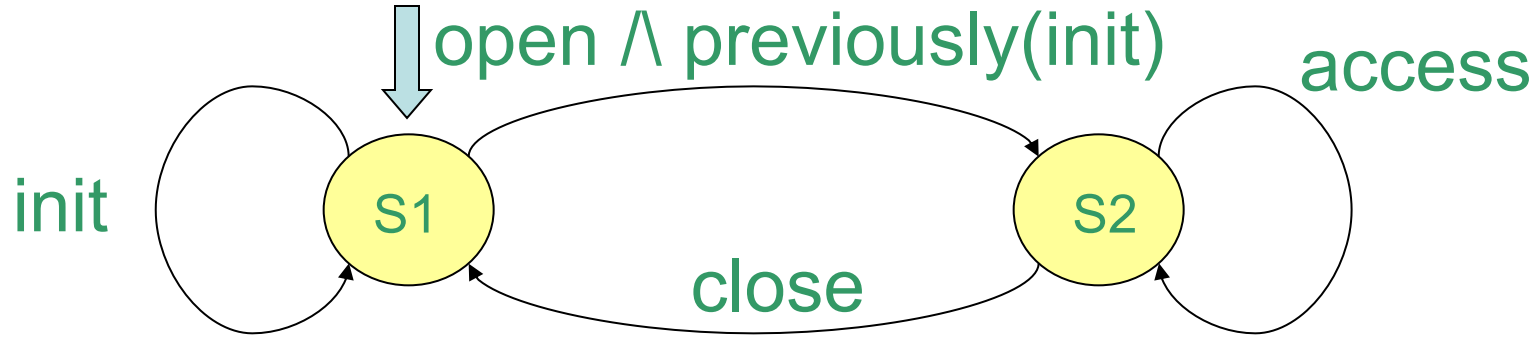

```
max S1() = init -> @ S1()
         /\ open -> @ S2()
         /\ ~(access \/ close) .
min S2() = access  -> @ S2()
         /\ close -> @ S1()
         /\ ~(init \/ open) .
mon M = S1() .
```

Timed Automata




```
max S1() = init -> @ S1()
         /\ open -> @ S2(clock)
         /\ ~(access \/ close) .
min S2(long t) =
             clock <= t+10
         /\ access -> @ S2()
         /\ close -> @ S1()
         /\ ~(init \/ open) .
```

Combining Automata and Temporal Logic




```
max S1() = init -> @ S1()
         /\ open -> (previously(init)
                     /\ @ S2())
         /\ ~(access \/ close) .
min S2() = access  -> @ S2()
         /\ close -> @ S1()
         /\ ~(init \/ open) .
mon M = S1() .
```



Regular Expressions

Property:
File accesses are always enclosed by open and close operations.

M = (init*;open;access*;close)*

```
max S(Term t) = t /\ @ S(t) . // Star
min P(Term t) = t /\ @ S(t) . // Plus


mon M =
  S(S(init());open();S(access());close()) .
```

KESTREL TECHNOLOGY

Grammars

Property:
Locks are acquired and released nested.

lock
release

lock
  lock
  release
release

```
// Match rule:

max match (Term l, Term r) =
  empty() \/ (l;match(l,r);r;match(l,r)) .


// Monitor:

mon M = match(lock(),release()) .
```

KESTREL TECHNOLOGY

Implementation Idea

- For each new event, the monitored formula is evaluated, resulting in either:
  - True - it is satisfied
  - False – it is violated
  - A formula in conjunctive normal form that the rest of the trace must satisfy.
- Then all formulas appearing after # (previous state) are evaluated and remembered for the next state.

Example Evaluation

Trace:

x	y	z
1	0	0
2	0	0
3	1	0
4	2	0
6	0	1

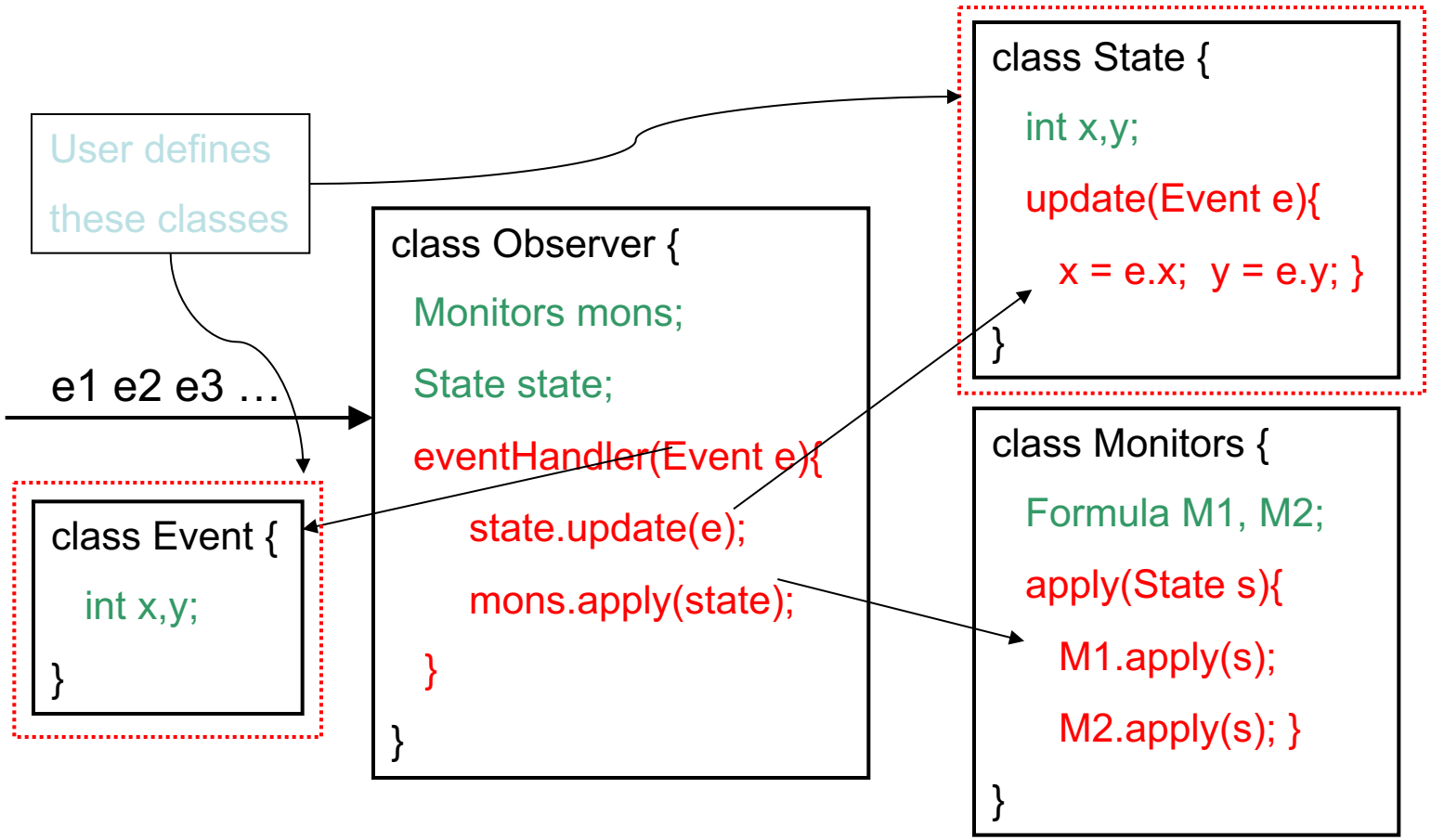
```
min R(int k) = Prev(x==k) /\ Even(z==k) .
mon M = Alws(y>0 -> R(y)) .

Init   : mon M = Alws(y>0 -> R(y))
         pas Prev(x==k) = false
1 0 0  : mon M = Alws(y>0 -> R(y))
         pas Prev(x==k) = k==1
2 0 0  : mon M = Alws(y>0 -> R(y))
         pas Prev(x==k) = k==1 \/ k==2
3 1 0  : mon M = Alws(y>0 -> R(y)) /\ Even(z==1)
         pas Prev(x==k) = k==1 \/ k==2 \/ k==3
4 2 0  : mon M = Alws(y>0 -> R(y)) /\ Even(z==1)/\ Even(z==2)
         pas Prev(x==k) = k==1 \/ k==2 \/ k==3 \/ k==4
6 0 1  : mon M = Alws(y>0 -> R(y)) /\ Even(z=2)
         pas Prev(x==k) = k==1 \/ k==2 \/ k==3 \/ k==4 \/ k==6
```

END ***** Property M is weakly violated: Even(z=2) pending**

KESTREL TECHNOLOGY

Putting it Together



KESTREL TECHNOLOGY

Adding Temporal Logic to JML

```
class User implements PriorityQueueUser {
  private List buffer = new ArrayList();
  private Map  priorities = new HashMap();
  private boolean conflicts = false;

  void add(Object argID){
    … implementation …
  }

  boolean contains(Object argID){
    … implementation …
  }

  Object next() throws PQException{
    … implementation …
  }
}
```

KESTREL TECHNOLOGY

The Interface is Now Specified - First an Abstract State

```
interface PriorityQueueUser {
  /*@ model instance JMLValueSet entries;
    @ initially entries != null && entries.isEmpty();
    @*/

  /*@ instance invariant
    @    entries != null &&
    @    (\forall JMLType e; entries.has(e);
    @         e instanceof QueueEntry);
    @
    @*/
}
```

```
//@ refine "PriorityQueueUserModel.java-refined";
```

Refining the Inteface with Method Specifications

```
interface PriorityQueueUser {
  void add(Object argID);

  /*@ pure @*/ boolean contains(Object argID);

  /*@ normal_behavior
    @     requires
    @       !entries.isEmpty();
    @     ensures
    @       (\exists QueueEntry r;
    @           entries.has(r) && \result == r.iD;
    @           (\forall QueueEntry o;
    @             entries.has(o) && !(r.equals(o));
    @               r.priorityLevel >= o.priorityLevel
    @               && r.timeStamp < o.timeStamp ) );
    @
  */
  Object next() throws PQException;
}
```

KESTREL TECHNOLOGY

Adding Temporal Logic

```
interface PriorityQueueUser {

  /*@ behavior
    @    always(
    @      contains(x?) returns true =>
    @         between(add(x),next() returns x));
   */

  void add(Object argID);

  boolean contains(Object argID);

  Object next() throws PQException;
}
```

Using Java Like Syntax For Eagle Itself

```
// LTL rules:

interface LTL{
  max always(Term t){
    t && @ always(t);
  }
  …
}

// Monitors:

interface FileSystem{
  import LTL;
  …
  monitor M(){
    always(close(file?) => previously(open(file)));
  }
}
```

KESTREL TECHNOLOGY

But Properties are Hard to Formulate

To quote quite excellent NASA software engineer when asked what properties his system would have to satisfy:

"*I have absolutely no idea what properties this system should satisfy*".

KESTREL TECHNOLOGY

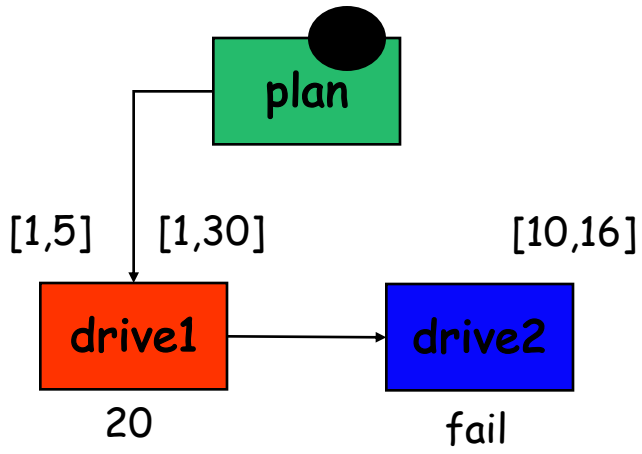
With: Corina Pasareanu
Willem Visser
Et al.

K9 Planetary Rover Executive



- Executive receives plans from a planner for direct execution
- Plan is a hierarchical structure of actions
- Multi-threaded system (35K lines of C++)

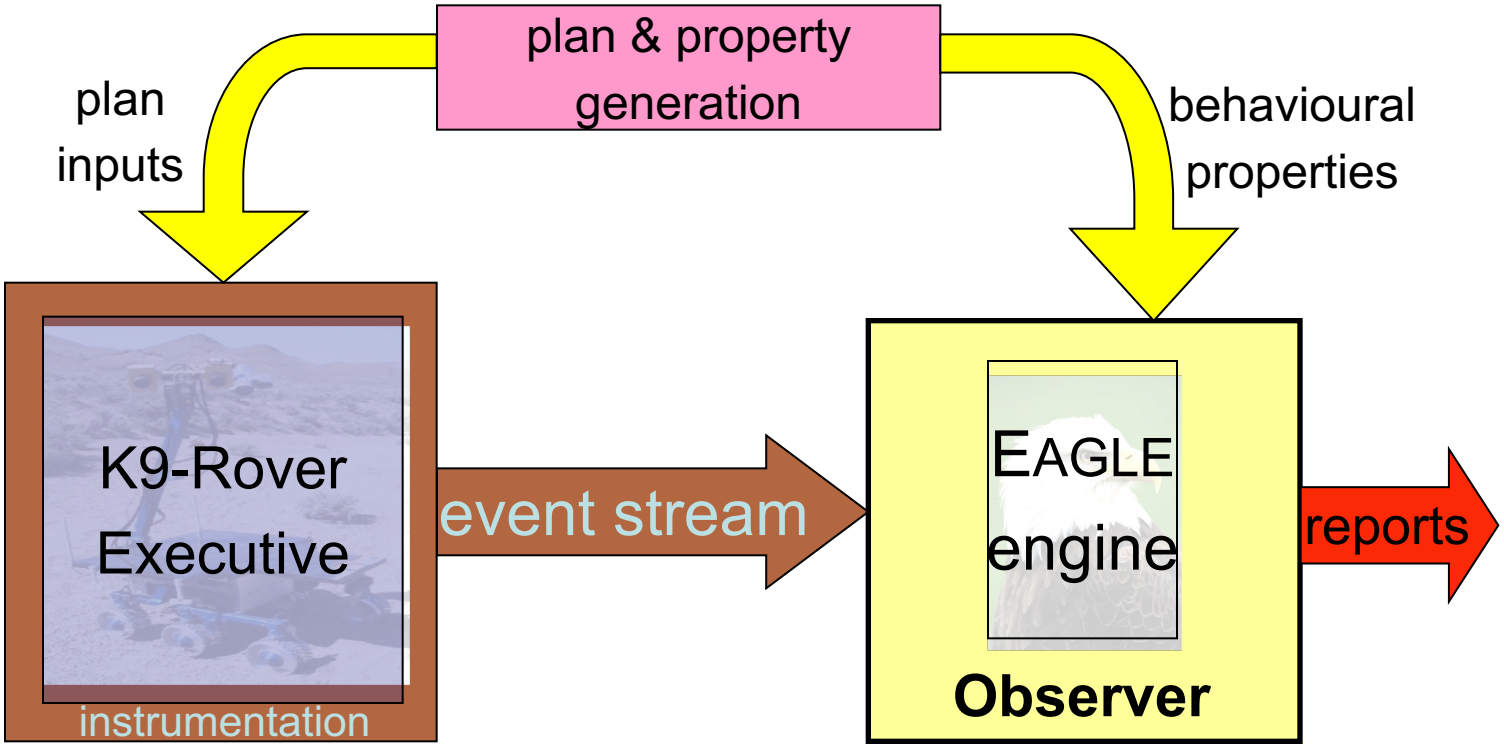
Example of Plan




```
(block :id plan
        :continue-on-failure
        :node-list (
            (task :id drive1
                :start-condition (time +1 +5)
                :end-condition (time +1 +30)
                :action BaseMove1)
            (task :id drive2
                :end-condition (time +10 +16)
                :action BaseMove2)
        )
)
```

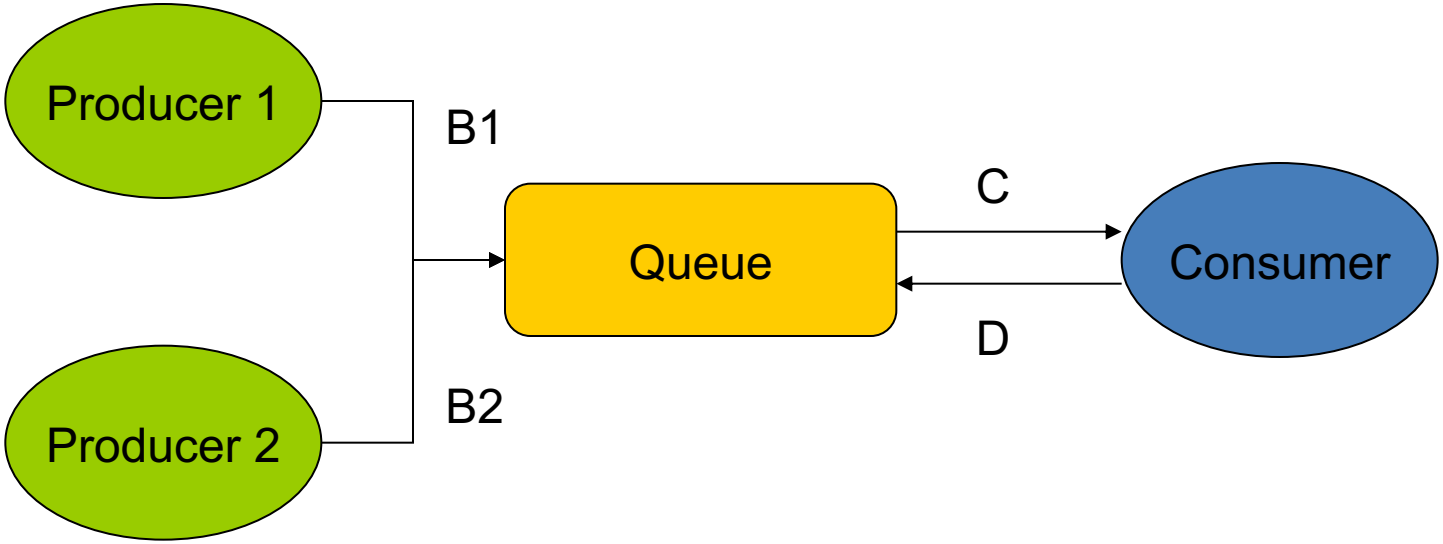
KESTREL TECHNOLOGY

Running X9



KESTREL TECHNOLOGY

A Priority Queue Example



KESTREL TECHNOLOGY

The State

```
class State extends EagleState {
  static String connector;
  static int    ident;
  static int    clock;

  static void update(String event) {...}

  static boolean B1(){return connector.equals("B1");}
  static boolean B2(){return connector.equals("B2");}
  static boolean C() {return connector.equals("C");}
  static boolean D() {return connector.equals("D");}
}
```

KESTREL TECHNOLOGY

Linear Temporal Logic

```
max A(Term t) = t /\ @ A(t)

min E(Term t) = t \/ @ A(t)

min P(Term t) = t \/ # P(t)

min U(Term t1, Term t2) =
        t2 \/ (t1 /\ @U(t1,t2))

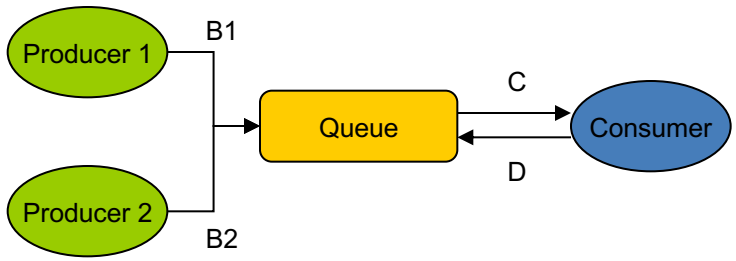
max W(Term t1, Term t2) =
        A(t1) \/ U(t1,t2)
```

Some Basic Definitions

```
min B12() = B1() ∨ B2()

min B2(int id) = B2() ∧ ident = id

min C(int id) = C() ∧ ident = id
```



KESTREL TECHNOLOGY

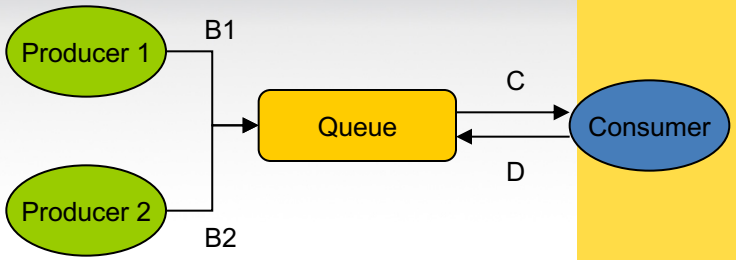
Extensions of E and P with Data Constraints

```
min Et(Term t, int time) =
        E(t /\ clock <= time)

min Eti(Term t, int time, int id) =
        E(t ∧ clock <= time /\ ident = id)

min Pi(Term t, int id) =
       P(t /\ ident = id)
```

Requirement #1



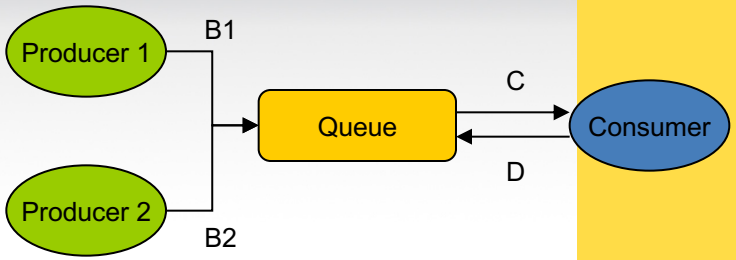
requiredResponse

Every message from a producer (communicated on either B1 or B2) must be responded to and acknowledged (on D) by the consumer within 50 seconds.

```
mon requiredResponse =
  A(B12() → Eti(D(), clock + 50, ident))
```

KESTREL TECHNOLOGY

Requirement #2



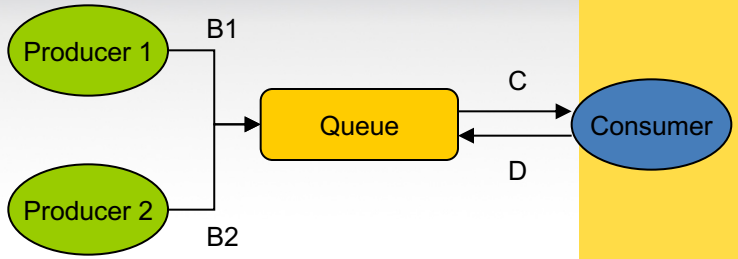
stayAliveQueue

Whenever a message is sent to the queue from a producer (on either B1 or B2), a message (not necessarily the same) must be consumed by the consumer (on C) within 32 seconds.

```
mon stayAliveQueue =
  A(B12() → Et(C(), clock + 32))
```

KESTREL TECHNOLOGY

Requirement #3



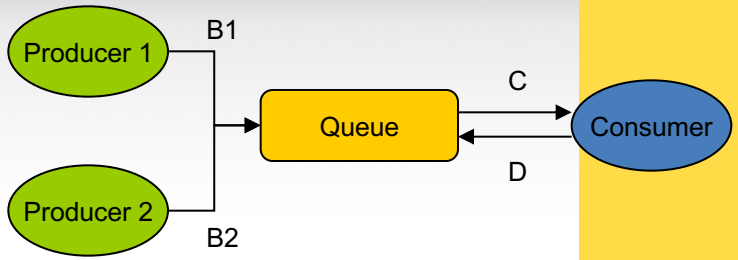
limitedSize

There are never more that 40 messages in the queue.

```
max CountSize(int size) =
  B12() → (size < 40 ∧                    @ CountSize(s
  C() → CountSize(size - 1) /\
  D() → CountSize(size)

mon limitedSize = CountSize(0)
```

Requirement #4



C D Alternation

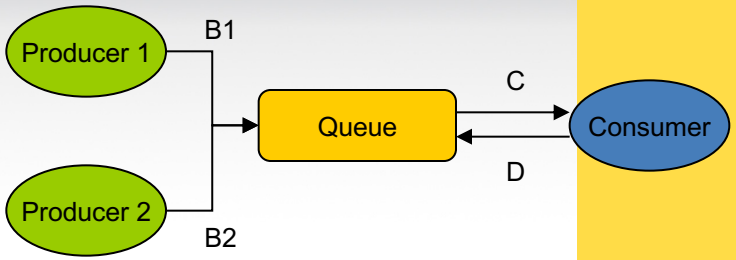
The consumer should alternate between consuming messages (on C) and acknowledging messages (on D).

```
max S1() = (C()  -> @S2()) /\
           (~C() -> @S1()) /\
           ~D()

Min S2() = (D()  -> @S1()) /\
           (~D() -> @S2()) /\
           ~C()

mon C_D_Alternation = S1()
```

Requirement #5



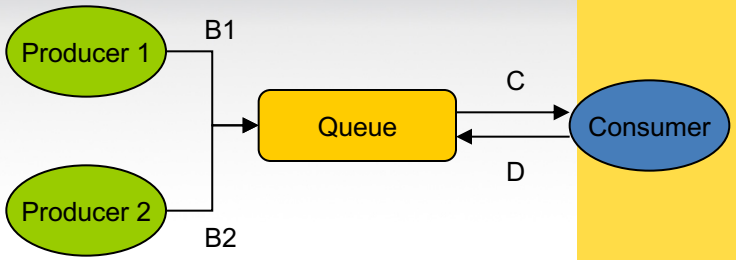
stayAliveConsumer

Every message consumed (on C) by the consumer is processed and acknowledged (on D) within 30 seconds.

```
mon stayAliveConsumer =
  A(C() → Et(D(), clock + 30))
```

KESTREL TECHNOLOGY

Requirement #6

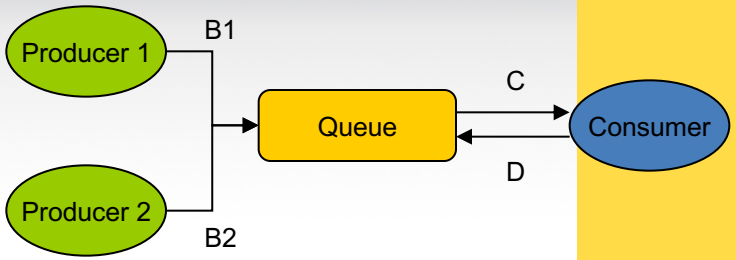


noJunkConsumed

Every message consumed by the consumer (on C) has previously Been produced (on B1 or B2).

```
mon noJunkConsumed =
  A(C() → Pi(B12(), ident))
```

Requirement #7

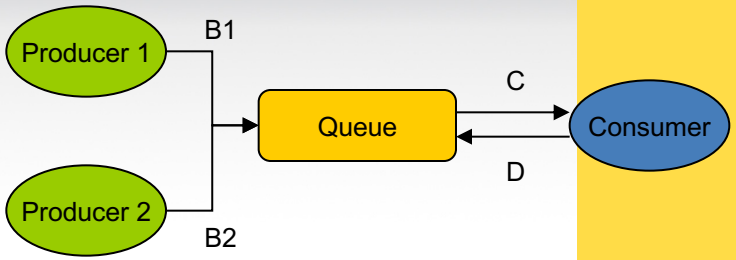


noJunkAcknowledged

Every message processed and acknowledged by the consumer (on D) has previously been consumed by the consumer (on C).

```
mon noJunkAcknowledged =
  A(D() -> Pi(C(), ident))
```

Requirement #8



orderPreserved

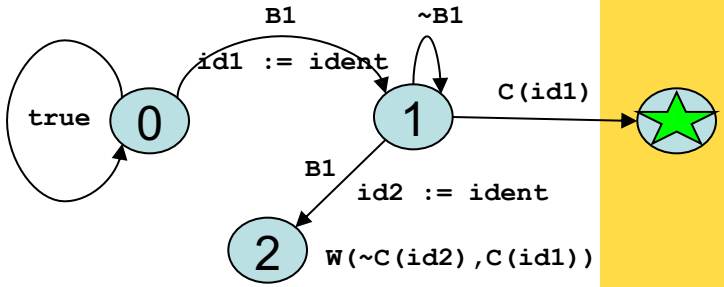
The queue behaves as a FIFO queue wrt. messages produced by producer 1. That is, the consumer consumes (on C) messages produced on B1 in the order in which they were produced.

```
max R0() = (B1() -> R1'(ident,0)) /\ @R0()

max R1'(int id1,int id2) =
  C(id1) \/
    ((B1() -> R2'(id1,ident)) /\
     (~B1() -> @R1(id1,id2)))

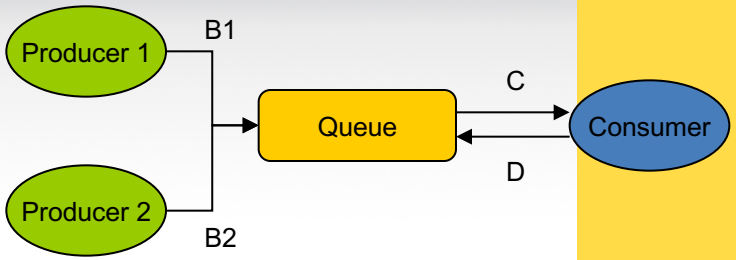
max R2'(int id1,int id2) = @R2(id1,id2)
max R2(int id1,int id2) = W(~C(id2),C(id1))

mon orderPreserved = R0()
```



KESTREL TECHNOLOGY

Requirement #9



B1HasPriority

Messages produced by producer 1 (on B1) have priority over messages produced by producer 2 (on B2). That is, as long as there are pending B1 messages in the queue, no B2 message will be consumed (on C) by the consumer.

```
min CForB1BeforeCForB2(int id) =
  U(~(C() /\ identFromB2(ident)), C(id))

min identFromB2(int id) = P(B2(id))

mon B1HasPriority =
  A(B1() -> CForB1BeforeCForB2(ident))
```

KESTREL TECHNOLOGY

Algorithm-Based Runtime Verification

- Deadlocks
  - Dining philosopher scenarios
- Data Races
  - Low-level data races on single variables
  - High-level data races on groups of variables
  - Flow data races, detecting out-of-date copies

KESTREL TECHNOLOGY

Conclusions

- We need good case studies. Utility is not proven.
- Is RV a study of languages or of algorithms?
- Strong Connections to
  - AOP, Test Oracles, Fault protection
- Can form basis for extending "Programming by contract" (Eiffel, JML) with trace assertions as the Jass system does.

KESTREL TECHNOLOGY

Temporal PointCuts in AOP

- Longer term ambition would be to extend a system like AspectJ with temporal point cuts:

"if the temporal formula F is violated then execute this piece of code."



The end